新版解答：学习冲分与控制退化的两条路线

已重新下载并逐项比对修改后的题面。主结果采用实际g++ C++14编译、公开参考引擎和真实决策计时；不沿用此前只计算I/O与Miss的旧Score结论。

最终608例 / 607例正BASE	学习冲分版	平衡版
几何平均代价降低	29.33%	25.10%
单组公式得分均值 / 满分10	2.239	1.676
最坏VAL/BASE	19,712.42倍	1.741倍
实现	21节点训练决策树选择轻量策略	安全Build、观测代价与短间隔读回反馈
交付文件	MySpillScheduler.cpp	MySpillScheduler_balanced.cpp

当前选择

若以新版Level 1的平均得分潜力为首要目标，学习冲分版在最终测试中更好；若更重视限制已见退化，选平衡版。两者都不是全输入最优解，学习版仍存在约1.97万倍的严重反例，不能只展示平均收益。

证据规模

四轮共生成2,432个新验证文件，另用998个历史开发输入训练；累计记录121,353次策略运行。最终608例用实际公开参考引擎、三个程序、各三次运行完成5472次对照。307个玩具规模精确模型检查及三组各500万操作的大输入也已完成。

所有数字都是本机构造负载结果，不是组委会隐藏数据成绩。单组公式均值没有套用50分总上限，不能由此宣布官方Level 1满分或专家评审得分。

题面变化决定了旧方案必须重做

项目	新版要求 / 对实现的影响
Score	I/O + miss_penalty×Miss + Total_Decision_Time。决策越重，越可能抵消I/O与Miss收益。
Level 1	单组clamp(10×(1-VAL/BASE),0,10)，总和上限50；与旧版50×BASE/VAL不同。
在线信息	只能用SystemState、其中的一条next_op和自行累积的历史；禁止预读输入或访问op_lines。
提交接口	无参数实例化MySpillScheduler；仅一个.cpp；不得修改引擎、主函数或数据结构。
编译与依赖	g++ -std=c++14 -O2；提交代码仅使用标准库。模型训练可在本地做，推断不依赖训练库。
可行性	最终每个分区<=M，且保证存在合法调度；旧版“分区大于M”的诊断不再属于正式合法输入。
动作约束	明确禁止落盘当前INSERT目标；BUILD不可LOAD；跨算子SPILL允许；每次读写独立收费。
时间与样例	单例墙上时间上限60秒；样例N已从10改为11；决策时间示例只是示意值。

核对来源

本次读取日期为2026年10月26日；网页updateTime对应2026-10-20T17:56:27+08:00。完整HTML、正文、公开响应、来源摘要和逐行差异均保存在本次研究目录。

官方第20题页面

保留的接口问题

文字说明称框架自身不计时，但参考timedSchedulerCall的lambda内部调用getSystemState，状态快照的构造/析构实际在计时区间内。本次保留参考实现，不把快照移出计时区间来降低分数。文字还称当前INSERT目标SPILL会被忽略，给出的executeActions未见相应过滤；两个提交都主动排除当前目标，因此不依赖该差异。

能看见什么，不能把什么偷偷当作输入

可用信息	合法使用方式
M、已用内存、W/R、K/P	构造容量缺口、读写代价与分区规模特征。
各分区大小、驻留状态、阶段	候选victim、跨算子共享内存、可加载性检查。
全局Hit/Miss与累计I/O字节	形成Probe时钟，感知整体命中情况；不能恢复每次Hit的分区。
next_op及参数	只知道紧邻下一条操作，可一步预取或保护下一目标。
自己发出的SPILL与后续Miss	记录落盘时间、短间隔读回，作为反复换入换出的反馈。

miss_penalty没有出现在SystemState中

新版还要求std::make_unique<MySpillScheduler>()无参数构造。因此旧版把真实miss_penalty传入构造函数的用法不能直接提交。两个新文件都不读输入流，也不从框架内部获取该值。

代价策略中的32微秒是固定启发式尺度，不是真实miss_penalty；涉及下一目标保护的保守比较仅使用题面保证的下界penalty>=1。N、B、build_count和probe_count也未作为隐藏参数传给调度器。

历史频率是受限观测，不是全量访问统计

只有Miss及回调中可见的next Probe能够定位分区。按全局Probe时钟去重并做惰性衰减；没有在命中时增设回调，没有将未观察到的Hit当成零访问，也没有声称恢复了真实频率。

两个版本都只返回调度动作，实际Probe仍按输入顺序执行。Miss回调中的提前LOAD下一分区满足末尾LOAD当前目标，并已由公开参考引擎实际执行验证；未使用更远未来。

学习冲分版：用小模型选择轻量策略

运行时流程

第一次需要作出淘汰决策时，从只读状态提取16个特征；内嵌的21节点、深度4决策树选择一种策略。之后按所选规则工作，运行过程中无需Python、NumPy或外部模型文件。树推断最多4次阈值比较，每个输入只选择一次。

候选策略	Build / Probe行为
最大分区与空间预取	缺口优先由最大分区覆盖；空闲空间可预取下一Probe。
小覆盖方案	优先选择足以覆盖缺口的较小单分区，减少即时写字节。
代价密度	写费 + 观测保留价值×(读费+固定尺度)，除以分区大小排序；比较单分区覆盖，去掉多余victim。
观测近期性	Build使用最大分区；Probe偏向淘汰较久未被合法观测到的分区。
安全Build + 代价Probe	避免在未知未来热点时过度依赖小分区Build淘汰。
观测热度/大小	以衰减的观测热度和大小确定保留优先级，保护已知下一目标。

为什么训练，而不只手写阈值

不同带宽、分区大小和混合阶段下，不同规则的得失不同。训练标签来自同一合成负载上各策略的实际Score；模型学习选择规则，而不是背诵某个输入的动作序列。输入中没有文件名、随机种子或完整未来。

风险来自Build预测仍可能猜错

该模型可能选中小覆盖Build规则。若写快读慢，少写一点却淘汰未来热点，随后读回可能远比写费昂贵。最终测试最坏约19712倍退化真实保留；新版单组公式把未优于基线的输入记0分，不以负分惩罚，所以平均得分与尾部风险必须分开看。

平衡版：
安全构建、代价与反馈结合

Build阶段不猜远期热点

优先淘汰最大的驻留分区并排除当前INSERT目标。若一个分区足够，单次扫描直接选出；只有缺口需要多个victim时才排序。

Probe阶段使用观测代价

分区保留价值由Miss与合法next Probe样本形成，按Probe时钟衰减。估计损失为 C=s/W+q×(s/R+32)，再按C/s选候选并比较单个覆盖方案。这里q是启发式保留权重，未经过概率校准。

短间隔读回反馈

记录自己对PROBE阶段分区发出SPILL的时刻。如果多次在max(64,2KP)个Probe内读回，在第三次且已知相邻两目标可共同驻留时，将Probe淘汰切换为观测热度/大小规则。这用于逃离“省一点写费却不断换入换出”的循环。

它是反馈信号，不是反事实最优证明：容量争用本来也可能导致短间隔读回，因此不能把每次返回都称为确定的误淘汰。

预取保持低额外开销

当前Miss计划释放空间后，如果下一目标可在不增加SPILL的情况下共同容纳，先LOAD下一目标，最后LOAD当前目标。周期回调仅在空闲空间足够时LOAD确定的下一Probe。

一次历史反例上的实际修复

第一轮的capacity_cycle_15中，固定代价策略产生约203.60亿微秒I/O；加入短间隔读回反馈后的相同输入回归约3.67亿。该结果用于解释机制，不作为独立新验证数据。不同规则仍可能在其他序列上变差。

Cao与Irani：GreedyDual-Size，USITS 1997

代价/大小与老化思想来自缓存研究；本题缺少完整Hit通知、Build会增长且重复SPILL收费，因此没有直接继承GDS的理论保证。

训练与四轮迭代：模型没有被强制当成赢家

轮次	实际工作与结论
第一轮	998个历史开发输入比较七种轻量规则；学习21节点选择器。平均收益较好，但新608例仍出现约544倍退化。
第二轮	限制Build候选，加入自己的短间隔读回反馈。新608例上学习路由最坏约1.88倍，平均几何代价下降约27.29%。
第三轮	推迟到第一次真实Miss作策略选择。得到7节点树，但没有稳定超过固定自适应专家，故保留为研究候选。
最终验证	冻结学习冲分版和平衡版后，再生成608例。随后以实际GNU参考引擎重算主结果，两个候选不再调参。

训练目标与数据分离

训练按同源种子分组，键别名与ID变换样例不跨训练/开发验证。第一轮有967个可提取决策特征的输入，774例拟合、193例选择模型；第三轮首次Miss特征有944例，757/187划分。

优化损失采用min(VAL/BASE,1)并尝试加入截断退化惩罚；开发验证目标还惩罚较大退化和节点数量。搜索深度、最小叶样本数和风险系数共60个组合。真正运行时只执行已冻结规则。

不同轮采用不同新种子，百分比不能直接相减当成同一数据集上的净改进；最终两候选的比较使用同一批608例、同一GNU基线。

Vietri等：ML-based LeCaR，HotStorage 2018

Song等：Learning Relaxed Belady，NSDI 2020

前者启发组合不同缓存专家，后者强调学习缓存中的开销控制。本实现是本题受限回调下的代价敏感选择器和自有淘汰账本，没有复现它们的完整算法，也没有迁移其论文收益或保证。

验证集覆盖与防止结果选择偏差

最终验证组成	具体覆盖
480个多分布输入	20类负载，每类24个新种子；其中一半随机重命名算子/分区ID。
128个端点组合	K、P、M、W、R、penalty、B各两个端点，完整2^7组合；全部随机重命名ID。
访问分布	固定/迁移热点、大小热点、突发、Zipf、顺序与循环扫描、热点夹杂冷扫描。
容量与阶段	两热点能/不能共同容纳、临界字节、空分区、接近M、多算子及Build/Probe交叠。
附加正确性	307个小状态精确下界、7个历史回归、隐藏惩罚值变换、真实引擎接入。
最大规模	三组完整500万操作：B=1全驻留、扫描热点混合、4GiB构建增长。

冻结与完整记录

每轮系统熵、种子、输入SHA-256、模型源文件SHA-256及参数均保存。最后一轮是在两个候选冻结后生成；最终审核已核对所有新输入及当前提交源文件摘要。开发结果、历史反例、各轮验证分开保存。

模型推断看不到的东西，训练时也不能伪装成特征

Score标签可使用自行生成的完整合成负载来评估策略；提交时只保留从合法状态到策略的映射，不读取实际比赛文件。额外把输入中的penalty从1改成1000，两个值下平衡版的Hit/Miss、读写字节、动作数量与选择结果保持一致。

小规模下界只用于检查，不用于在线提交

所有307个小输入的被测代价都不低于独立驻留集合动态规划给出的离线最优值。该离线模型允许知道完整未来；没有把“未低于下界”误写成“在线算法达到最优”。玩具内存规模也未混入正式范围性能主表。

最终主结果：实际GNU参考引擎，三次中位数

指标	公开基线	学习冲分版	平衡版
几何平均VAL/BASE	1.000000	0.706662	0.749044
单组公式均值 / 10	0	2.239	1.676
胜 / 平 / 负	-	361/60/186	333/83/191
P95相对代价	1	1.388	1.139
最坏相对代价	1	19,712.42	1.741
总Miss	294,516	238,926	191,355
累计决策时间 / 秒	0.506	0.483	0.359

主集608例，其中1例BASE实际为0，不参与比值；表中607例均使用自身同编译器的BASE。单组得分均值按新版公式计算，没有把608组得分总和的50分上限当作“官方满分”证据。

几何平均的合成分布重采样区间

学习冲分版：95%区间 0.6500 – 0.7720。

平衡版：95%区间 0.7066 – 0.7918。

区间仅描述本次负载混合和种子抽样，不保证隐藏数据。总决策时间包含参考计时区间中的状态快照；CPU-only输入对计时分辨率、调度和缓存状态敏感。主指标按每例三次中位数计算，三次I/O和Hit/Miss完全一致。

为什么保留两个提交

新版Level 1不会对VAL>=BASE额外扣负分，因此冲分版可用更多成功样例的改善换取部分输入记0分。但Level 2还评价极端场景稳定性，且隐藏分布未知；平衡版的尾部结果更好。没有一个版本在全部指标和全部输入上同时占优。

编译器与引擎：不能混用不同BASE

实际编译验证

g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0；编译选项含-std=c++14 -O2，并增加-Wall -Wextra -pedantic-errors检查。两个提交都通过；唯一警告来自公开朴素类未使用的state参数。提交文件不包含main，只依赖标准库。

实际公开引擎，而非只验证译写模型

下载的参考代码仅插入候选调度器并替换make_unique实例类型，除此以外逐字核验一致。最终608例×3程序×3次=5472次都运行该引擎；三组最大规模也运行相同引擎。

跨平台差异已经量化

对照	最终608例中I/O或Miss不同的输入数
公开朴素基线：Windows与GNU	14
平衡版：独立回放与GNU实际引擎	0
学习冲分版：独立回放与GNU实际引擎	0

公开基线按分区大小排序，同大小分区没有完整平局规则；不同标准库的std::sort可以作出不同合法顺序。本轮发现14例BASE的I/O或Miss不同。因此主表全部重算为GNU参考引擎结果，没有拿Windows的BASE配GNU的VAL。

同环境重复仍会有计时波动

std::chrono度量的是调用区间的经过时间；系统调度和缓存状态会影响很短的回调。数据保存每次原始值，统计使用中位数；不删除较慢运行，也不把本机耗时承诺成评测机保证。

Lykouris与Vassilvitskii：Competitive Caching with Machine Learned Advice，ICML 2018

预测与鲁棒后备相结合是可参考的研究路线；本实现没有获得该论文模型下的竞争比，也没有把实验最坏1.741倍当作理论上界。

三组500万操作：真实参考引擎全部跑完

场景	版本	VAL/BASE	决策秒	进程秒
全驻留 / B=1	公开朴素基线	1.0000	1.437	3.88
全驻留 / B=1	平衡版	1.0139	1.457	3.17
全驻留 / B=1	学习冲分版	0.9589	1.378	3.15
扫描、热点与迁移	公开朴素基线	1.0000	2.559	4.79
扫描、热点与迁移	平衡版	0.7704	6.193	8.00
扫描、热点与迁移	学习冲分版	0.7672	7.090	8.85
构建增长 / M=4GiB	公开朴素基线	1.0000	0.001	2.85
构建增长 / M=4GiB	平衡版	1.1886	0.002	2.25
构建增长 / M=4GiB	学习冲分版	0.8213	0.002	2.26

每个输入均有完整元数据及Build、Switch、Probe，N=5,000,000、K=8、P=64。最长进程约8.85秒，已包含解析、模拟与决策；本机各例均在60秒内完成。

退化也完整展示

4GiB增长输入中，平衡版Score高于基线；全驻留输入没有I/O和Miss差异，主要反映计时噪声及回调开销。它们没有被删除，也没有被混合负载上的改善掩盖。

I/O是按照字节数与带宽计算的模拟代价，不会真的等待相应磁盘时间。参考引擎记录分区大小，没有分配真实4GiB哈希数据；不能由进程内存较小宣称实际数据库获得同等内存压缩。

容量安全可证明，全输入最优尚不能证明

预测错误不影响容量合法性

设当前内存使用U，上限M，当前INSERT分区大小a，待插入b。新版保证最终分区<=M，所以a+b<=M。溢出缺口d=U+b-M，满足d<=U-a：排除当前分区后，其余驻留字节仍足够覆盖缺口。各策略都会累计victim直到释放足够空间。

Probe Miss目标大小s<=M，所需缺口U+s-M<=U；若保护另一个驻留目标大小t，先检查s+t<=M，因此缺口<=U-t，仍有足够的其他victim。

联合加载只在释放空间后U-freed+s+t<=M时使用。先SPILL、再LOAD下一目标、最后LOAD当前目标；加载过程中占用单调增加到不超过M的最终值。周期预取直接检查目标大小<=空闲容量。

这些条件与模型选择或热点预测是否准确无关。K×P<=512，索引范围和64位字节计数适配题面上限；所有动作保持整分区读写与阶段约束。

复杂度与实际边界

周期回调O(1)，持久观测数组O(KP)且上限固定；特征构造只在首次决策做一次O(KP)。Miss/多victim的最坏复杂度O(KP log(KP))；单个最大分区足够的Build场景直接O(KP)。没有在线求解器、深度模型或超预算搜索。

尚未解决的最优性问题

未知的未来访问、缺失的真实miss_penalty、不可见的逐次Hit以及不同负载分布，限制了预测的可靠性。训练集和隐藏数据可能不同；学习版的巨大反例证明平均收益不能替代稳健性。当前交付是经过多轮选择与验证的候选方案，不是所有在线算法中的最优性证书。

功能合法性、Score改善与专家评审是不同层次。本次没有官方隐藏数据和专家打分，因此不宣称已获Level 0/1/2/3对应的官方分数。

交付与复现入口

文件 / 目录	用途
design/MySpillScheduler.cpp	学习冲分版，单文件提交候选；构造时无须参数。
design/MySpillScheduler_balanced.cpp	平衡版，另一份同名类实现；两者选择一个提交。
iteration_20261026/topic_20_current.*	新版公开题面快照；problem_diff.txt记录旧新差异。
iteration_20261026/round1、round2、round3	历史策略、训练报告、冻结记录与各轮验证指针。
iteration_20261026/train_and_evaluate.py	七策略开发对照、代价敏感决策树、分组验证及新种子输入生成。
iteration_20261026/gcc_validation.py	g++编译及公开引擎接入、完整500万操作测试。
iteration_20261026/gcc_full_holdout.py	最终608例在实际公开引擎上的三程序三次运行。
results/final_audit.json	源文件/数据摘要、合规静态检查、全部主结果与核验摘要。
holdout_20261026_111448_c4e556/gcc_published_engine_results.jsonl	主表的5472条原始GNU引擎运行结果。
holdout_20261026_111448_c4e556/manifest.json	最终608个输入的完整路径、种子、参数与摘要。

复现

本机已有Python+NumPy、Visual Studio及Ubuntu g++。Windows可运行build.cmd及train_and_evaluate.py test重跑独立回放；Ubuntu运行gcc_validation.py与gcc_full_holdout.py重跑实际引擎。extra_validation.py small完成精确下界、回归和隐藏参数不变性检查；final_audit.py核验交付。

提交时只选一个实现文件。评测系统提供SpillScheduler、SystemState、Action及主程序；不需要提交训练脚本、模型文件或改动执行引擎。完整路径和命令见本次README.txt。

题面来源、研究借鉴、自己设计的规则、训练选择、实测结果和未证明部分均已分开说明。后续迭代应保留尾部反例，使用新的冻结后数据检验改动，尤其不能只凭平均收益把任一候选称为全局最优。